

CHƯƠNG 6: THỰC NGHIỆM HYBRID SEARCH VỚI TẬP DỮ LIỆU SCIFACT

6.1 Tóm tắt

Thực nghiệm này trình bày quy trình triển khai và đánh giá hiệu suất của các phương pháp truy xuất thông tin (Information Retrieval) trên tập dữ liệu khoa học SciFact. Bằng việc tận dụng cơ sở dữ liệu Oracle 26ai, hệ thống tích hợp và so sánh ba phương pháp: Truy xuất ngữ nghĩa (Semantic Search), Truy xuất từ khóa (Keyword Search) và phương pháp lai (Hybrid Search). Kết quả thực nghiệm trên 300 truy vấn cho thấy Hybrid Search đạt hiệu suất cao nhất với chỉ số Hit@10 đạt 82.33% và Hit@50 đạt 92.00%, chứng minh tính hiệu quả của việc kết hợp từ vựng và ngữ nghĩa trong hệ thống RAG chuyên ngành khoa học.

6.2 Giới thiệu

Trong bối cảnh lượng tài liệu nghiên cứu khoa học tăng trưởng theo cấp số nhân, việc xác minh các luận điểm khoa học (scientific claim verification) trở thành một bài toán cấp thiết. Bài toán này đòi hỏi các hệ thống xử lý ngôn ngữ tự nhiên (NLP) không chỉ hiểu ngữ nghĩa chuyên sâu mà còn phải truy xuất chính xác các tài liệu làm bằng chứng (evidence abstracts).

Quá trình truy xuất thường gặp khó khăn do sự phức tạp của từ vựng chuyên ngành và các biến thể đồng nghĩa. Để giải quyết vấn đề này, Oracle 26ai Database cung cấp nền tảng AI Vector Search cho phép thực hiện tìm kiếm ngữ nghĩa và tìm kiếm văn bản (Full-text search) trên cùng một hệ quản trị cơ sở dữ liệu duy nhất. Nghiên cứu này triển khai một pipeline truy xuất toàn diện trên tập dữ liệu SciFact nhằm đánh giá và tìm ra chiến lược tối ưu nhất để trích xuất bằng chứng khoa học cho các mô hình Retrieval-Augmented Generation (RAG). Toàn bộ mã nguồn thực nghiệm ở <https://github.com/24520152-ben/Hybrid-Search-SciFact-Oracle-26ai-Database>.

6.3 Cơ sở lý thuyết

6.3.1 Tổng quan về tập dữ liệu SciFact

Khái niệm và Quy mô

SciFact là một tập dữ liệu chuyên biệt được thiết kế để phục vụ bài toán xác minh luận điểm khoa học (scientific claim verification) tự động. Tập dữ liệu này bao gồm 1.409 luận điểm khoa học được gán nhãn thủ công bởi các chuyên gia, đi kèm với một kho ngữ liệu quy mô gồm 5.183 tài liệu tóm tắt (abstracts). Nhiệm vụ đặt ra cho các mô hình là phải truy xuất được các tài liệu chứa bằng chứng và phân loại xem tài liệu đó Hỗ trợ (SUPPORTS) hay Bác bỏ (REFUTES) luận điểm đầu vào.

Đặc điểm của Luận điểm (Scientific Claims)

Điểm khác biệt cốt lõi làm nên giá trị của SciFact so với các tập dữ liệu xác minh thông tin thông thường (ví dụ: FEVER) nằm ở tính tự nhiên của dữ liệu. Trong khi FEVER sử dụng các câu hỏi nhân tạo (synthetic claims) được tạo ra bằng cách biến đổi văn bản trên Wikipedia, các luận điểm trong SciFact là các khẳng định nguyên tử (atomic claims) được trích xuất trực tiếp từ các câu trích dẫn (citation sentences - citances) xuất hiện trong các

bài báo khoa học thực tế. Quy trình này đảm bảo rằng các chủ đề trong SciFact phản ánh chính xác ngôn ngữ, tính phức tạp và các thuật ngữ chuyên ngành của văn bản nghiên cứu.

Cấu trúc Bằng chứng (Rationales) và Dán nhãn

Với mỗi cặp “luận điểm - tài liệu”, hệ thống phân loại sẽ gán một trong ba nhãn: Hỗ trợ (SUPPORTS), Bác bỏ (REFUTES), hoặc Không có thông tin (NOINFO). Tuy nhiên, bài toán không dừng lại ở việc dán nhãn mức tài liệu. Để đảm bảo tính minh bạch (interpretable), bất kỳ nhãn SUPPORTS hoặc REFUTES nào cũng phải được chứng minh bằng ít nhất một tập hợp bằng chứng (rationale). Bằng chứng được định nghĩa là một tập hợp tối thiểu các câu trích xuất từ tài liệu, mà khi kết hợp lại, chúng cung cấp đủ tiền đề để một chuyên gia có thể kết luận tính đúng/sai của luận điểm. Trong SciFact, mỗi bằng chứng là một khối thông tin súc tích chứa tối đa 3 câu.

Quy trình xây dựng kho ngữ liệu và Tài liệu nhiễu (Distractor Abstracts)

Kho ngữ liệu của SciFact được xây dựng dựa trên nguồn dữ liệu S2ORC, tiến hành chọn lọc ngẫu nhiên từ các tạp chí khoa học có mức độ uy tín cao (như Cell, Nature, JAMA, BMJ). Nhằm mô phỏng bài toán tìm kiếm thông tin thực tế và tăng tính thử thách cho các mô hình truy xuất (Retrieval), nhóm tác giả đã bổ sung thêm các “tài liệu nhiễu” (distractor abstracts). Đây là các tài liệu được trích dẫn cùng trong một bài báo với tài liệu chuẩn, có sự tương đồng rất lớn về mặt chủ đề và từ vựng, nhưng lại không chứa bằng chứng trực tiếp để giải quyết luận điểm. Chính sự xuất hiện của các tài liệu nhiễu này khiến các phương pháp truy xuất từ khóa truyền thống (Keyword Search) gặp khó khăn, tạo ra động lực thực tiễn để ứng dụng các kỹ thuật thấu hiểu ngữ nghĩa sâu như Hybrid Search hoặc Semantic Search trong miền tri thức khoa học.

6.3.2 Kỹ thuật Phân mảnh Văn bản (Text Chunking / Splitting)

Khái niệm và Vai trò

Trong quá trình xây dựng các hệ thống truy xuất thông tin và RAG, phân mảnh văn bản (text chunking) là kỹ thuật chia nhỏ các tài liệu gốc có dung lượng lớn thành các phân đoạn (chunks) nhỏ hơn. Quá trình này đóng vai trò thiết yếu nhằm đảm bảo hai mục tiêu: giúp từng phân đoạn có thể được trích xuất một cách độc lập và đảm bảo kích thước của đoạn văn bản đầu vào nằm trong giới hạn cho phép của cửa sổ ngữ cảnh (context window) của các mô hình ngôn ngữ. Để tối ưu hóa quá trình này, các thư viện như LangChain cung cấp nhiều chiến lược phân mảnh khác nhau, mỗi chiến lược sở hữu những ưu điểm riêng biệt để phù hợp với từng định dạng dữ liệu đầu vào.

Các chiến lược phân mảnh cốt lõi

- **Phân mảnh dựa trên cấu trúc văn bản (Text structure-based):** Văn bản tự nhiên luôn được tổ chức theo các đơn vị phân cấp như đoạn văn, câu và từ. Chiến lược này tận dụng chính cấu trúc phân cấp đó để tạo ra các phân đoạn nhằm duy trì mạch văn tự nhiên, tính gắn kết ngữ nghĩa nội bộ và thích ứng linh hoạt với độ chi tiết của tài liệu. Thuật toán tiêu biểu cho phương pháp này là **RecursiveCharacterTextSplitter**. Thuật toán này hoạt động theo cơ chế đệ quy: ưu tiên giữ nguyên cấu trúc của các đơn vị văn bản lớn (như đoạn văn); trong trường hợp kích thước một đơn vị vượt ngưỡng **chunk_size** quy định, thuật toán mới tiếp tục chia nhỏ xuống cấp độ tiếp theo (như câu) và sẽ tiếp tục cho đến cấp độ từ nếu cần thiết. Đây được xem là chiến lược mặc định tối ưu nhất cho hầu hết các bài toán vì nó cân bằng xuất sắc giữa việc bảo toàn ngữ cảnh và quản lý kích thước phân đoạn.

- **Phân mảnh dựa trên độ dài (Length-based):** Đây là cách tiếp cận trực quan nhất, trong đó hệ thống sẽ cắt văn bản hoàn toàn dựa trên giới hạn kích thước định trước, đảm bảo tính nhất quán về kích thước phân đoạn và dễ dàng tinh chỉnh theo yêu cầu của từng mô hình. Chiến lược này thường được triển khai dưới hai hình thức:
 - + **Dựa trên số lượng Token (Token-based):** Đo lường và cắt văn bản theo số lượng token, đặc biệt hữu ích khi tương tác trực tiếp với các mô hình ngôn ngữ lớn (LLMs).
 - + **Dựa trên số lượng ký tự (Character-based):** Sử dụng các thuật toán như `CharacterTextSplitter` để chia nhỏ văn bản dựa trên một số lượng ký tự cố định, mang lại sự đồng nhất cao giữa nhiều định dạng văn bản khác nhau.
- **Phân mảnh dựa trên cấu trúc tài liệu (Document structure-based):** Đối với các tài liệu mang cấu trúc đánh dấu rõ ràng như HTML, Markdown, JSON hoặc mã nguồn (Code), việc phân mảnh dựa trên cấu trúc nội tại của tài liệu mang lại hiệu quả vượt trội. Thay vì cắt văn bản một cách cơ học, hệ thống nhóm các văn bản có liên quan về mặt ngữ nghĩa thông qua các thẻ cấu trúc. Ví dụ: tài liệu Markdown sẽ được chia dựa trên các thẻ tiêu đề (`#`, `##`), tệp JSON phân chia theo các đối tượng/mảng, và mã nguồn được tách theo các hàm hoặc lớp logic. Việc bảo tồn tổ chức logic nguyên bản giúp cải thiện đáng kể hiệu suất cho các tác vụ hạ nguồn như truy xuất (retrieval) hoặc tóm tắt (summarization).

6.3.3 Kỹ thuật Biểu diễn Vector (Vector Embedding)

Khái niệm và Nguyên lý hoạt động

Mô hình biểu diễn vector (Embedding models) là cốt lõi của các hệ thống truy xuất thông tin hiện đại. Nhiệm vụ của mô hình này là chuyển đổi văn bản thô (có thể là một câu, một đoạn văn, hoặc toàn bộ tài liệu) thành một vector số học có số chiều cố định. Cấu trúc vector này được thiết kế để nắm bắt và codộng ý nghĩa ngữ nghĩa (semantic meaning) của văn bản.

Nhờ việc ánh xạ ngôn ngữ tự nhiên vào không gian toán học, máy tính có khả năng tìm kiếm và so sánh văn bản dựa trên ngữ nghĩa thay vì phải khớp chính xác từng từ khóa (exact-match). Trong không gian này, các văn bản chứa các ý tưởng tương đồng sẽ được định vị gần nhau. Quá trình này vận hành qua hai giai đoạn chính:

- **Mã hóa vector (Vectorization):** Mô hình học sâu sẽ mã hóa chuỗi đầu vào thành một vector trong không gian đa chiều.
- **Tính toán độ tương đồng (Similarity scoring):** Các phép toán hình học được sử dụng để đo lường khoảng cách giữa các vector, từ đó xác định mức độ liên quan về mặt ngữ nghĩa giữa các văn bản.

Các thước đo độ tương đồng (Similarity Metrics)

Để đánh giá độ gần gũi giữa các vector, hệ thống thường sử dụng ba thước đo toán học cơ bản:

- **Khoảng cách Euclidean (Euclidean distance):** Đo lường khoảng cách theo đường thẳng trực tiếp giữa hai điểm trong không gian vector.

- **Tích vô hướng (Dot product):** Đánh giá mức độ hình chiếu của vector này lên vector kia, kết hợp cả hướng và độ lớn của vector.
- **Độ tương đồng Cosine (Cosine similarity):** Đây là thước đo phổ biến nhất trong xử lý ngôn ngữ tự nhiên, đo lường góc giữa hai vector độc lập với độ dài của chúng. Độ tương đồng cosine giữa vector truy vấn $\mathbf{q}$ và vector tài liệu $\mathbf{d}$ được định nghĩa bởi công thức: $\text{cosine_similarity}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|}$

Các đặc trưng cấu trúc của Mô hình Embedding

Khi triển khai mô hình biểu diễn vector, có một số tham số kỹ thuật trọng yếu ảnh hưởng trực tiếp đến hiệu năng của hệ thống RAG:

- **Số chiều không gian (Dimensionality):** Kích thước của vector tỷ lệ thuận với khả năng biểu diễn thông tin nhưng đồng thời cũng tiêu tốn nhiều tài nguyên lưu trữ và tính toán hơn. Kích thước phổ biến dao động từ 384 chiều (các mô hình nhỏ như `all-MiniLM-L6-v2`), 768 chiều (`all-mpnet-base-v2`) cho đến trên 3072 chiều ở các mô hình thương mại cỡ lớn.
- **Giới hạn ngữ cảnh (Context length):** Các mô hình Transformer truyền thống thường bị giới hạn ở mức xử lý 512 token cho mỗi chuỗi đầu vào. Tuy nhiên, các mô hình tiên tiến hiện nay (như `BGE-M3` hay các mô hình text-embedding thế hệ mới) có năng lực xử lý lên đến 8192 token, mở ra khả năng mã hóa toàn vẹn các đoạn tài liệu dài.
- **Tiền tố đặc thù (Query and document prompts):** Một số mô hình mã nguồn mở hiện đại (như `E5`, `BGE`) được huấn luyện tối ưu hóa bằng cách sử dụng các tiền tố (prefix) khác nhau phân biệt giữa chuỗi truy vấn (query) và chuỗi tài liệu (passage). Việc thiết lập đúng các tiền tố này ở pha trích xuất là bắt buộc để duy trì chất lượng của mô hình.

Tích hợp trong kiến trúc LangChain

Framework LangChain cung cấp giao diện trừu tượng Embeddings cho phép chuẩn hóa việc giao tiếp với các mô hình ngôn ngữ từ nhiều nhà cung cấp khác nhau. Giao diện này cung cấp hai cơ chế xử lý tách biệt:

- **embed_documents:** Được tối ưu để mã hóa song song hàng loạt tài liệu trong pha xây dựng cơ sở dữ liệu (Ingestion).
- **embed_query:** Được tinh chỉnh để mã hóa một truy vấn đơn lẻ từ phía người dùng trong thời gian thực.

Đồng thời, để tối ưu hóa chi phí tính toán (đặc biệt khi gọi API từ các mô hình thương mại) và giảm độ trễ, kiến trúc hỗ trợ module `CacheBackedEmbeddings` giúp lưu trữ tạm thời (cache) các vector đã được tính toán dựa trên mã băm (hash) của văn bản thông qua một hệ thống lưu trữ byte (ByteStore).

6.3.4 Các Phương pháp Truy xuất Thông tin: Semantic Search và Keyword Search

Trong lĩnh vực truy xuất thông tin (Information Retrieval), việc thấu hiểu và đáp ứng chính xác nhu cầu của người dùng phụ thuộc rất lớn vào công nghệ tìm kiếm nền tảng. Hiện

nay, hai phương pháp tiếp cận cốt lõi đang được ứng dụng rộng rãi là Truy xuất Từ khóa (Keyword/Lexical Search) và Truy xuất Ngữ nghĩa (Semantic Search). Mỗi phương pháp sở hữu các cơ chế, ưu điểm và không gian ứng dụng riêng biệt.

Truy xuất Từ khóa (Keyword Search / Lexical Search)

- **Khái niệm và Nguyên lý:** Truy xuất từ khóa là phương pháp tìm kiếm truyền thống, hoạt động dựa trên cơ chế so khớp chuỗi ký tự (string matching). Hệ thống sẽ tìm kiếm các tài liệu chứa chính xác các từ hoặc cụm từ khóa (keywords) mà người dùng nhập vào. Cơ chế này tương tự như việc tra cứu mục lục ở cuối một cuốn sách để tìm các trang chứa một thuật ngữ cụ thể.
- **Quy trình hoạt động:**
 - + **Lập chỉ mục (Indexing):** Hệ thống quét toàn bộ kho tài liệu và tạo ra một cấu trúc dữ liệu, thường là chỉ mục đảo ngược (inverted index), nhằm ánh xạ mỗi từ vựng với các tài liệu chứa nó.
 - + **Xử lý và Truy xuất (Query Processing & Retrieval):** Khi nhận truy vấn, hệ thống tìm kiếm chính xác các từ khóa trong tập chỉ mục và trích xuất các tài liệu thỏa mãn.
 - + **Xếp hạng (Ranking):** Các tài liệu được xếp hạng mức độ liên quan dựa trên các số liệu thống kê như tần suất xuất hiện của từ khóa (Keyword frequency / TF-IDF), vị trí của từ khóa (ví dụ: nằm ở tiêu đề hay thân bài), và độ uy tín của tài liệu.
- **Đặc điểm và Ứng dụng:** Phương pháp này nổi bật ở tốc độ xử lý nhanh và độ chính xác tuyệt đối đối với các truy vấn rõ ràng. Do đó, nó cực kỳ hiệu quả trong các bài toán truy xuất dữ liệu có cấu trúc hoặc tìm kiếm định danh cụ thể:
 - + Phân tích log hệ thống (tìm kiếm mã lỗi, địa chỉ IP).
 - + Truy vấn cơ sở dữ liệu dựa trên mã định danh (SKU sản phẩm, mã người dùng).
 - + Khám phá dữ liệu pháp lý (e-discovery) nhằm đảm bảo không bỏ sót bất kỳ văn bản nào chứa thuật ngữ pháp lý chính xác.
- Tuy nhiên, giới hạn lớn nhất của Keyword Search là sự thiếu hụt khả năng thấu hiểu ngôn ngữ. Hệ thống gặp khó khăn khi đối mặt với các từ đồng nghĩa, từ đa nghĩa, và đặc biệt là không nắm bắt được ý định thực sự (user intent) đằng sau các truy vấn có tính chất nhập nhằng.

Truy xuất Ngữ nghĩa (Semantic Search)

- **Khái niệm và Nguyên lý:** Khắc phục những hạn chế của tìm kiếm từ khóa, Truy xuất Ngữ nghĩa là một công nghệ tìm kiếm nâng cao, sử dụng Trí tuệ Nhân tạo (AI) và Xử lý Ngôn ngữ Tự nhiên (NLP) để phân tích ngôn ngữ theo cách hiểu của con người. Thay vì so khớp chuỗi ký tự, hệ thống tập trung vào việc giải mã ý định, sắc thái và ngữ cảnh bao hàm trong câu truy vấn để thiết lập mối quan hệ giữa các từ vựng và các khái niệm trừu tượng.
- **Quy trình hoạt động:**
 - + **Nhận diện ý định (Understanding Intent):** Hệ thống phân tích mục tiêu cốt lõi của người dùng (ví dụ: tìm kiếm thông tin, mua sắm, hay điều hướng).

- + **Phân tích ngữ cảnh (Contextual Analysis):** Mô hình tích hợp các tín hiệu ngoại vi như vị trí địa lý, lịch sử tìm kiếm hoặc mốc thời gian. (Ví dụ: truy vấn “nhà hàng đang mở cửa” sẽ được xử lý kết hợp với dữ liệu định vị và thời gian thực).
- + **So khớp khái niệm (Concept Matching):** Thông qua các mô hình biểu diễn không gian vector (embeddings) và Đồ thị Tri thức (Knowledge Graphs), hệ thống nhận diện các thực thể và mối liên hệ giữa chúng thay vì lệ thuộc vào từ vựng bề mặt.
- + **Truy xuất và Xếp hạng:** Kết quả được xếp hạng dựa trên sự tương đồng về mặt ngữ nghĩa (contextual similarity) giữa truy vấn và tài liệu.
- **Đặc điểm và Ứng dụng:** Semantic Search mang lại trải nghiệm khám phá thông tin trực quan và mượt mà, lý tưởng cho các hệ thống phục vụ người dùng cuối với ngôn ngữ tự nhiên, đa dạng:
 - + **Thương mại điện tử:** Xử lý các truy vấn phức tạp mang tính hội thoại (ví dụ: “áo khoác nam ấm áp dưới 100 đô la”).
 - + **Hỗ trợ khách hàng:** Định tuyến chính xác câu hỏi của người dùng đến các tài liệu hỗ trợ ngay cả khi họ không sử dụng đúng thuật ngữ chuyên môn.
 - + **Khám phá nội dung & Y tế:** Gợi ý nội dung đa phương tiện dựa trên chủ đề, thể loại; hoặc truy xuất các hồ sơ bệnh án, nghiên cứu y khoa thông qua việc mô tả triệu chứng thay vì tên bệnh chính xác.

Đối sánh và Hướng tiếp cận Lai (Hybrid Search)

Sự khác biệt cốt lõi giữa hai phương pháp có thể được tổng hợp ở các khía cạnh sau:

- **Độ linh hoạt của truy vấn:** Trong khi Keyword Search đòi hỏi các thuật ngữ chính xác và rõ ràng, Semantic Search có khả năng xử lý linh hoạt từ đồng nghĩa, cách diễn đạt lại (paraphrasing) và câu hỏi tự nhiên.
- **Cơ chế xếp hạng:** Keyword Search phụ thuộc vào tần suất và sự so khớp chuỗi, trong khi Semantic Search đo lường khoảng cách trong không gian vector (embeddings).

Trong thực tiễn xây dựng các hệ thống trí tuệ nhân tạo hiện đại, việc sử dụng đơn lẻ một phương pháp thường không giải quyết triệt để mọi bài toán. Do đó, một kiến trúc tìm kiếm lai (Hybrid Search) thường được ưu tiên triển khai, kết hợp sức mạnh của việc so khớp chính xác từ vựng (để tìm các danh từ riêng, mã định danh) với trí thông minh nhận thức ngữ cảnh của vector hóa (để hiểu ý định và từ đồng nghĩa), từ đó tối đa hóa hiệu suất truy xuất thông tin.

6.3.5 Cấu trúc Chỉ mục HNSW (Hierarchical Navigable Small World)

Khái quát về Bài toán Tìm kiếm Lân cận Xấp xỉ (K-ANNS)

Trong các hệ thống truy xuất dựa trên vector (như Oracle AI Vector Search), việc tìm kiếm K-láng giềng gần nhất (K-Nearest Neighbor Search - K-NNS) bằng phương pháp duyệt toàn bộ (brute-force) có độ phức tạp tuyến tính $O(N)$, khiến nó trở nên bất khả thi đối với các tập dữ liệu quy mô lớn. Để giải quyết vấn đề này, bài toán Tìm kiếm Lân cận Xấp xỉ (K-ANNS) được áp dụng, cho phép đánh đổi một tỷ lệ nhỏ độ chính xác để gia tăng đột phá về tốc độ. Trong số các kỹ thuật K-ANNS, HNSW (Hierarchical Navigable Small World) nổi lên như một cấu trúc dữ liệu hoàn toàn dựa trên đồ thị mang lại hiệu suất tối tân (state-of-the-art).

Cấu trúc Đồ thị Phân tầng (Hierarchical Structure)

Khác với các đồ thị Navigable Small World (NSW) truyền thống dễ bị suy giảm hiệu suất ở không gian chiều thấp hoặc bị mắc kẹt ở các cực tiểu cục bộ (local minima), HNSW giải quyết vấn đề bằng cách phân tách các liên kết (links) theo độ dài và đặt chúng vào các tầng (layers) khác nhau.

Ý tưởng của HNSW có sự tương đồng lớn với cấu trúc danh sách nhảy xác suất 1D (probabilistic skip list) nhưng được tổng quát hóa bằng cách thay thế danh sách liên kết bằng các đồ thị tiệm cận (proximity graphs). Trong cấu trúc này:

- Mỗi phần tử dữ liệu khi được nạp vào hệ thống sẽ được cấp một mức tầng tối đa (maximum layer) một cách ngẫu nhiên, tuân theo phân phối xác suất suy giảm theo hàm mũ (exponentially decaying probability distribution).
- Nhờ phân phối này, các tầng trên cùng sẽ rất thưa thớt (chỉ chứa các liên kết dài), trong khi các tầng thấp hơn (đặc biệt là tầng 0 - ground layer) sẽ chứa toàn bộ các phần tử với các liên kết ngắn, dày đặc.

Quy trình Truy xuất (Greedy Search Routing)

Quá trình tìm kiếm vector trong HNSW được thực hiện lặp đi lặp lại từ trên xuống dưới theo nguyên lý tham lam (greedy search).

1. Pha Thu phóng (Zoom-in phase): Quá trình truy xuất luôn bắt đầu từ tầng cao nhất (top layer), nơi chỉ chứa các liên kết dài nhất. Thuật toán duyệt qua đồ thị một cách tham lam để hướng dòng tìm kiếm về phía vector truy vấn (query vector).
2. Chuyển tầng: Thuật toán tiếp tục duyệt trên tầng hiện tại cho đến khi chạm đến một cực tiểu cục bộ (local minimum) – tức là không tìm thấy điểm nào trên tầng đó gần với truy vấn hơn điểm hiện tại.
3. Tại điểm cực tiểu cục bộ này, thuật toán sẽ đi xuống tầng thấp hơn (với các liên kết ngắn hơn) và sử dụng chính điểm cực tiểu ở tầng trên làm điểm bắt đầu (enter-point) cho tầng dưới.
4. Quá trình này lặp lại cho đến khi thuật toán kết thúc quá trình tìm kiếm ở tầng 0 (ground layer).

Nhờ cấu trúc phân tầng này, giới hạn số lượng kết nối tối đa của mỗi phần tử trên các tầng là một hằng số, giúp độ phức tạp của quá trình tìm kiếm đạt mức logarit $O(\log(N))$.

Chiến lược Chọn lọc Láng giềng (Neighbor Selection Heuristic)

Một đóng góp kỹ thuật quan trọng của HNSW là cơ chế chọn lọc láng giềng khi khởi tạo liên kết. Thay vì thiết lập kết nối một cách ngây thơ với các điểm gần nhất (simple nearest neighbors), HNSW áp dụng một thuật toán heuristic có tính toán đến khoảng cách giữa chính các ứng viên láng giềng nhằm tạo ra các liên kết đa dạng hóa (diverse connections).

Cụ thể, thuật toán đánh giá các ứng viên theo thứ tự từ gần đến xa so với phần tử gốc. Một kết nối mới chỉ được tạo ra nếu ứng viên đó nằm gần phần tử gốc hơn so với bất kỳ ứng viên nào khác đã được kết nối trước đó. Heuristic này giúp thuật toán mô phỏng được đồ thị láng giềng tương đối (relative neighborhood graph), đảm bảo đồ thị luôn giữ được tính liên kết toàn cục (global connectivity), đặc biệt hiệu quả trong việc chống lại hiện tượng phân cụm dữ liệu dày đặc (highly clustered data). Do đó, hệ thống không bị mắc kẹt tại ranh giới của các cụm dữ liệu cục bộ trong quá trình truy xuất.

6.3.6 Tích hợp Oracle AI Vector Search và LangChain

Tổng quan về Oracle AI Vector Search

Oracle AI Database hỗ trợ các luồng công việc Trí tuệ Nhân tạo (AI) bằng cách cho phép truy vấn dữ liệu dựa trên ý nghĩa (ngữ nghĩa) thay vì chỉ dựa trên so khớp từ khóa. Nền tảng này kết hợp khả năng tìm kiếm ngữ nghĩa trên dữ liệu phi cấu trúc với khả năng lọc quan hệ trên dữ liệu nghiệp vụ trong cùng một hệ quản trị duy nhất. Kiến trúc này giúp các nhà phát triển xây dựng các quy trình truy xuất (chẳng hạn như hệ thống RAG) mà không cần triển khai thêm một cơ sở dữ liệu vector độc lập, từ đó ngăn chặn tình trạng phân mảnh dữ liệu trên nhiều nền tảng lưu trữ.

Tích hợp LangChain thông qua OracleVS

Thông qua thư viện `langchain-oracledb`, module `OracleVS` đóng vai trò là một vector store tích hợp toàn diện. Tích hợp này hỗ trợ thực thi nhiều thao tác cốt lõi:

- Nạp tài liệu và biểu diễn vector (embeddings) trực tiếp vào cơ sở dữ liệu Oracle.
- Thực thi các phép tìm kiếm độ tương đồng (similarity search).
- Khởi tạo các chỉ mục vector chuyên dụng phục vụ truy xuất tốc độ cao. Hệ thống giao tiếp thông qua driver `python-oracledb`, hỗ trợ chế độ ‘Thin’ (kết nối trực tiếp không yêu cầu thư viện Oracle Client) và có thể chuyển sang chế độ ‘Thick’ để kích hoạt các tính năng cơ sở dữ liệu nâng cao.

Chiến lược Lập chỉ mục (Vector Indexing)

Để tối ưu hóa tìm kiếm vector, Oracle AI Vector Search hỗ trợ khởi tạo các loại chỉ mục xấp xỉ, nổi bật là cấu trúc đồ thị HNSW (Hierarchical Navigable Small World) và IVF (Inverted File). Các chỉ mục này được xây dựng dựa trên các chiến lược đo lường khoảng cách (distance metrics) khác nhau bao gồm Tích vô hướng (Dot Product), Độ tương đồng Cosine (Cosine) và Khoảng cách Euclidean. Quá trình tạo chỉ mục có thể được tinh chỉnh sâu thông qua các tham số hiệu suất như độ chính xác mục tiêu (accuracy), số luồng xử lý song song (parallel), hoặc các tham số kỹ thuật chuyên biệt như số lượng hàng xóm (neighbors) và efConstruction đối với mạng HNSW.

Cơ chế Lọc nâng cao (Advanced Filtering)

Oracle Database nâng cao năng lực tìm kiếm thông qua việc phân tầng ba cơ chế lọc dữ liệu:

- **Tiền lọc (Pre-filtering):** Áp dụng các bộ lọc SQL truyền thống để thu hẹp tập dữ liệu trước khi thực thi tìm kiếm vector, giúp giảm khối lượng dữ liệu cần xử lý và tối ưu hóa đáng kể thời gian truy vấn.
- **Lọc trong quá trình (In-filtering):** Tận dụng năng lực của AI Vector Search để lọc kết quả trực tiếp trong quá trình tính toán độ tương đồng bằng các thuật toán và chỉ mục tối ưu, đảm bảo duy trì độ chính xác ngữ nghĩa mà không cần quét toàn bộ dữ liệu.
- **Hậu lọc (Post-filtering):** Áp dụng các điều kiện SQL bổ sung nhằm tinh chỉnh kết quả sau khi tìm kiếm vector hoàn tất, phục vụ các logic nghiệp vụ cụ thể.

Hệ thống hỗ trợ đa dạng các toán tử lọc siêu dữ liệu (metadata filters) như `$eq`, `$gt`, `$in`, `$regex`, `hasSubstring` kết hợp cùng các toán tử logic định tuyến (`$and`, `$or`, `$nor`) để thực thi các khối truy vấn phức tạp.

Tìm kiếm Lai (Hybrid Search) và Tìm kiếm Toàn văn (Full-text Search)

- **Truy xuất Lai (Hybrid Search):** Trên phiên bản Oracle Database 26ai, nền tảng cung cấp khả năng tìm kiếm lai, kết hợp đồng thời tìm kiếm từ khóa (toàn văn) và tìm kiếm ngữ nghĩa (vector) vào một năng lực truy xuất duy nhất. Quá trình này được vận hành thông qua module `OracleHybridSearchRetriever` và yêu cầu khởi tạo `OracleVectorizerPreference` để đồng bộ hoàn toàn với cấu hình mô hình embedding. Kết quả trả về sẽ đánh giá mức độ phù hợp tổng hợp thông qua các điểm số thành phần là `text_score` và `vector_score`.
- **Truy xuất Toàn văn (Full-text Search):** Khả năng tìm kiếm từ khóa chính xác được đảm nhiệm bởi hệ thống Oracle Text thông qua module `OracleTextSearchRetriever`. Kiến trúc này cho phép khởi tạo chỉ mục `SEARCH INDEX` trên các cột văn bản, thực thi các truy vấn `CONTAINS` tích hợp cơ chế so khớp mờ (`fuzzy=True`) nhằm xử lý linh hoạt các trường hợp sai lệch chính tả. Ngoài ra, hệ thống hỗ trợ chế độ toán tử (`operator_search=True`) để nhận diện trực tiếp các biểu thức luận lý phức tạp của Oracle Text (như `NEAR`, `ABOUT`, `WITHIN`).

6.4 Phương pháp

Quy trình thực nghiệm được xây dựng bằng ngôn ngữ Python, sử dụng thư viện `LangChain` để tương tác trực tiếp với Oracle Database thông qua driver `oracledb` chạy ở chế độ Thin mode.

6.4.1 Data Ingestion & Preprocessing

Kho ngữ liệu gồm 5,183 tài liệu ban đầu từ corpus của SciFact được xử lý theo pipeline sau:

1. **Text Chunking:** Sử dụng thuật toán `RecursiveCharacterTextSplitter` với cấu hình `chunk_size = 500` và `chunk_overlap = 50`. Quá trình này chia nhỏ kho ngữ liệu thành 18,168 đoạn (chunks), mỗi đoạn được cấp một UUID ngẫu nhiên đồng thời lưu giữ `corpus_id` nguyên bản của tài liệu gốc.
2. **Vector Embedding:** Chuyển đổi toàn bộ các đoạn văn bản thành không gian vector mật độ cao bằng mô hình `sentence-transformers/all-MiniLM-L6-v2`. Quá trình mã hóa sử dụng chuẩn hóa kích thước vector (`normalize_embeddings = True`).
3. **Database Ingestion:** Dữ liệu được đẩy vào bảng `SciFact` trong Oracle Database theo từng batch với kích thước `BATCH_SIZE = 500`. Chiến lược tính khoảng cách được thiết lập là `DistanceStrategy.COSINE`.

6.4.2 Retrieval Pipelines

Nghiên cứu tiến hành xây dựng và đối chiếu ba chiến lược truy xuất để lấy ra top k tài liệu phù hợp nhất:

1. **Semantic Search (Truy xuất ngữ nghĩa):** Khởi tạo chỉ mục cấu trúc đồ thị HNSW trên cột chứa vector với tên `hnsw_scifact`. Hệ thống sử dụng `VectorStoreRetriever` để thực hiện tìm kiếm độ tương đồng (`search_type="similarity"`).
2. **Keyword Search (Truy xuất từ khóa):** Khởi tạo chỉ mục văn bản hệ thống Oracle Text mang tên `keyword_scifact`. Sử dụng thành phần `OracleTextSearchRetriever`

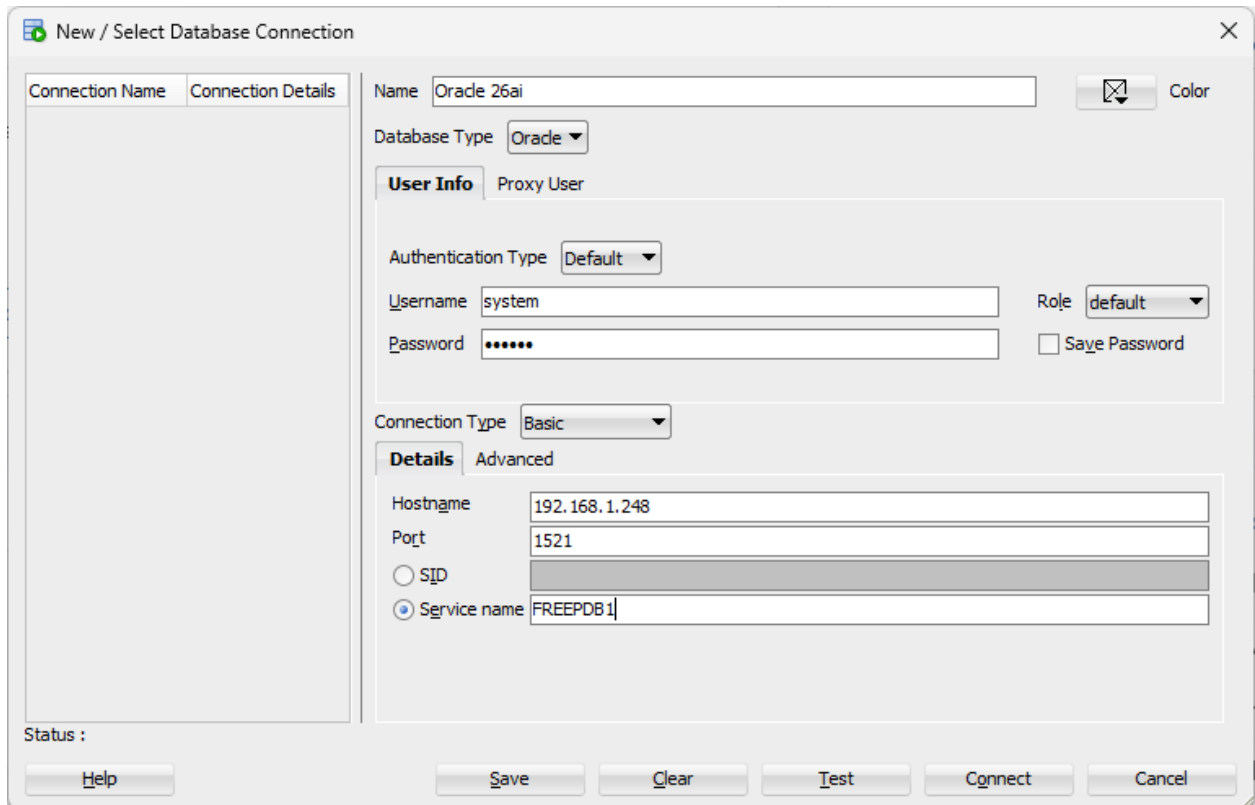
kết hợp tham số `fuzzy=True` nhằm kích hoạt tính năng khớp từ gần đúng, giúp sửa lỗi chính tả và nâng cao độ phủ.

3. **Hybrid Search (Truy xuất hỗn hợp):** Chiến lược lai kết hợp kết quả trả về của cả hai phương pháp trên. Hệ thống thực hiện lấy hợp (Union) danh sách các `corpus_id` trích xuất được từ Semantic Search và Keyword Search tại cùng một cấu hình ngưỡng k .

6.5 Thực Nghiệm & Kết quả

6.5.1 Đưa dữ liệu vào vector store thông qua OracleVS

Đầu tiên, mở Oracle 26ai Database thông qua VirtualBox và thiết lập kết nối với SQL Developer như hướng dẫn ở chương 3, tuy nhiên lần này không chọn kết nối với SID `free` mà chọn kết nối với Service name `FREEPDB1` như hình 6.1. `FREEPDB1` là tên dịch vụ mặc định (Service Name) của cơ sở dữ liệu có thể cắm (Pluggable Database - PDB) được tạo sẵn trong phiên bản Oracle Database Free. Đây là môi trường cơ sở dữ liệu độc lập bên trong CDB (Container Database), nơi tạo bảng, schema và lưu trữ dữ liệu.

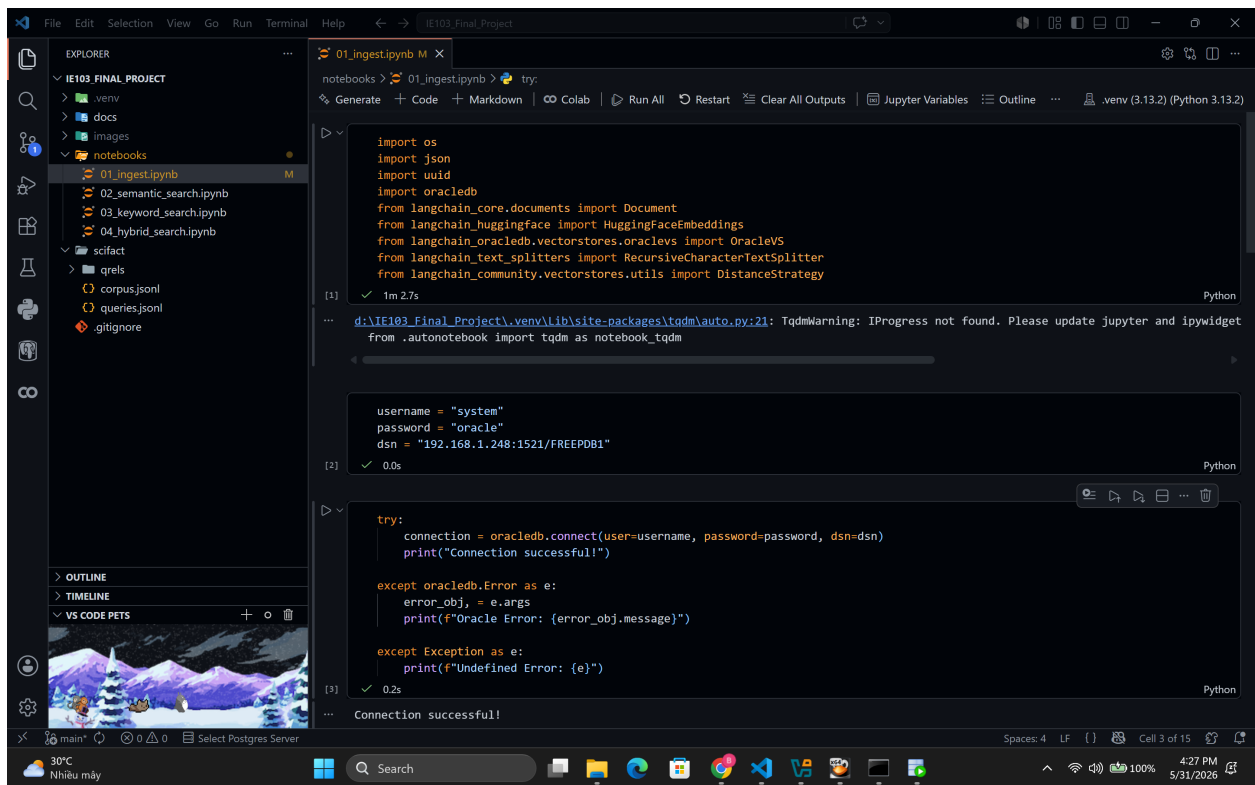


Hình 6.1: SQL Developer kết nối với Oracle 26ai Database với cấu hình như sau

Tiếp theo là thiết lập môi trường thực nghiệm trên notebook với ngôn ngữ Python. Ta phải tải các thư viện cần thiết thông qua lệnh sau:

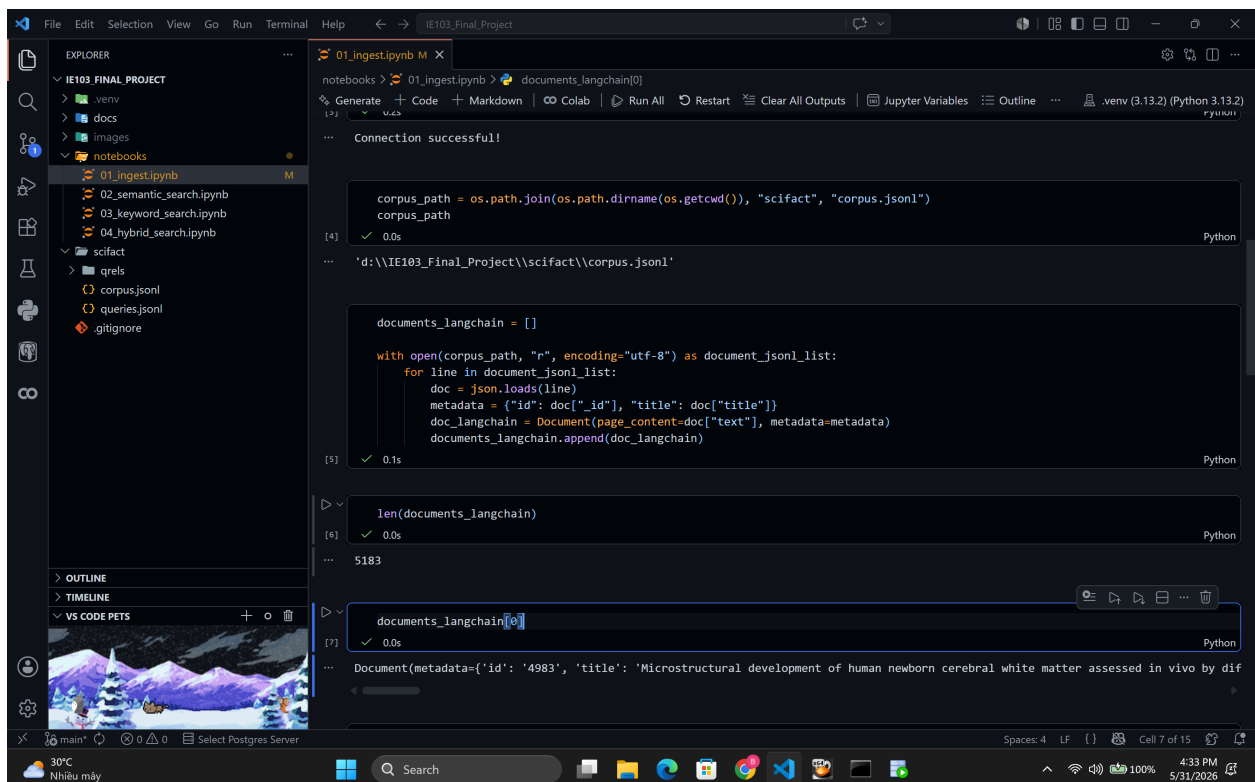
```
pip install langchain-oracledb langchain-huggingface langchain-text-splitters
```

Sau khi đã tải các thư viện cần thiết, ta import các thư viện và kết nối với Oracle 26ai Database như hình 6.2:



Hình 6.2: Notebook kết nối với Oracle 26ai Database như sau

Sau khi kết nối thành công, ta tiến hành chuyển 5183 tài liệu tóm tắt (abstracts) thành Document của Langchain như hình 6.3

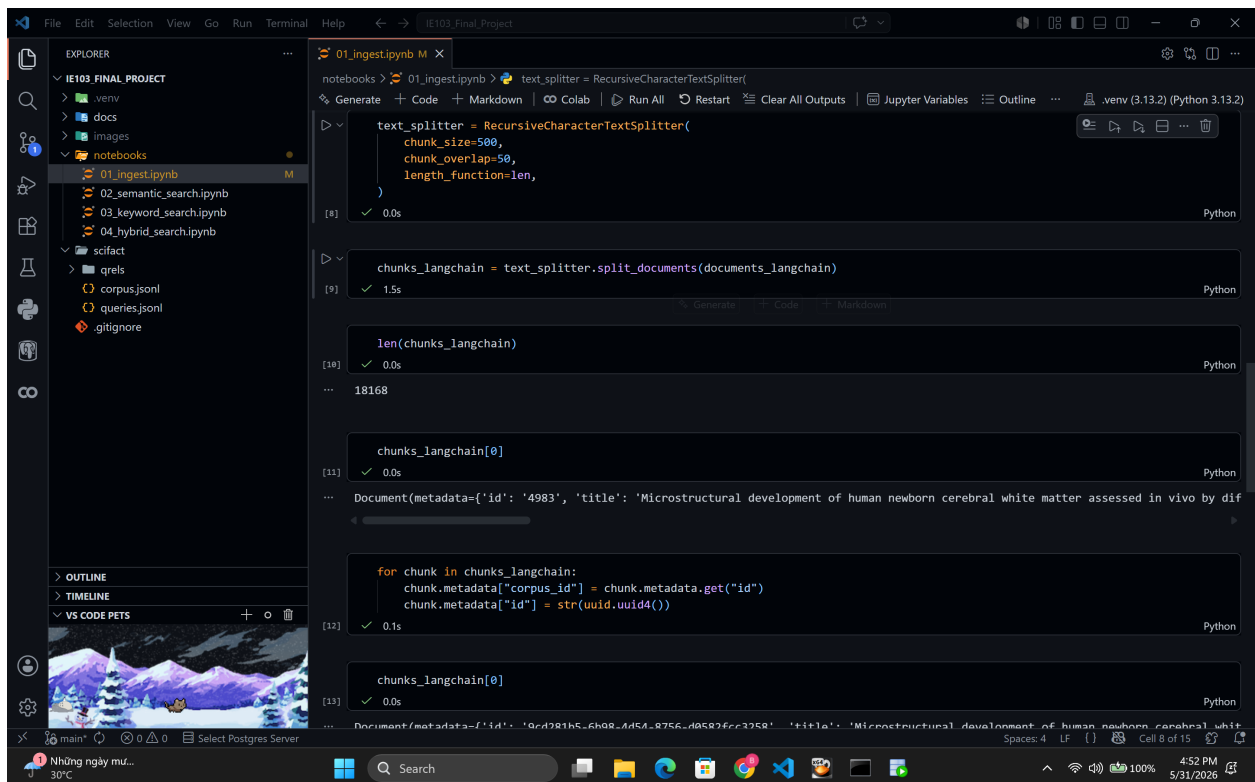


Hình 6.3: Chuyển 5183 tài liệu tóm tắt (abstracts) thành Document của Langchain

Một abstract sau khi chuyển sang Document sẽ có dạng như sau:

```
1 Document(metadata={'id': '4983', 'title': 'Microstructural development of
    ↳ human newborn cerebral white matter assessed in vivo by diffusion
    ↳ tensor magnetic resonance imaging.'}, page_content='Alterations of
    ↳ the architecture of cerebral white matter in the developing human
    ↳ brain can affect cortical development and result in functional
    ↳ disabilities. A line scan diffusion-weighted magnetic resonance
    ↳ imaging (MRI) sequence with diffusion tensor analysis was applied to
    ↳ measure the apparent diffusion coefficient, to calculate relative
    ↳ anisotropy, and to delineate three-dimensional fiber architecture in
    ↳ cerebral white matter in preterm (n = 17) and full-term infants (n =
    ↳ 7). To assess effects of prematurity on cerebral white matter
    ↳ development, early gestation preterm infants (n = 10) were studied a
    ↳ second time at term. In the central white matter the mean apparent
    ↳ diffusion coefficient at 28 wk was high, 1.8 microm2/ms, and
    ↳ decreased toward term to 1.2 microm2/ms. In the posterior limb of
    ↳ the internal capsule, the mean apparent diffusion coefficients at
    ↳ both times were similar (1.2 versus 1.1 microm2/ms). Relative
    ↳ anisotropy was higher the closer birth was to term with greater
    ↳ absolute values in the internal capsule than in the central white
    ↳ matter. Preterm infants at term showed higher mean diffusion
    ↳ coefficients in the central white matter (1.4 +/- 0.24 versus 1.15
    ↳ +/- 0.09 microm2/ms, p = 0.016) and lower relative anisotropy in
    ↳ both areas compared with full-term infants (white matter, 10.9 +/-
    ↳ 0.6 versus 22.9 +/- 3.0%, p = 0.001; internal capsule, 24.0 +/- 4.44
    ↳ versus 33.1 +/- 0.6% p = 0.006). Nonmyelinated fibers in the corpus
    ↳ callosum were visible by diffusion tensor MRI as early as 28 wk;
    ↳ full-term and preterm infants at term showed marked differences in
    ↳ white matter fiber organization. The data indicate that quantitative
    ↳ assessment of water diffusion by diffusion tensor MRI provides
    ↳ insight into microstructural development in cerebral white matter in
    ↳ living infants.')
```

Tuy nhiên, một abstract có thể khá dài và sẽ không thể embedding do vượt quá số lượng token đầu vào có thể xử lý được của mô hình embedding cũng như vượt quá context window khi đưa Document vào làm context cho LLMs ở bước Generation trong các mô hình RAG. Do vậy, cần phải chia nhỏ Document thành các Chunk để giải quyết các vấn đề trên. Một thuật toán khá phổ biến để Chunking chính là RecursiveCharacterTextSplitter, ta tiến hành chia nhỏ tài liệu với `chunk_size = 500` và `chunk_overlap = 50` như hình 6.4

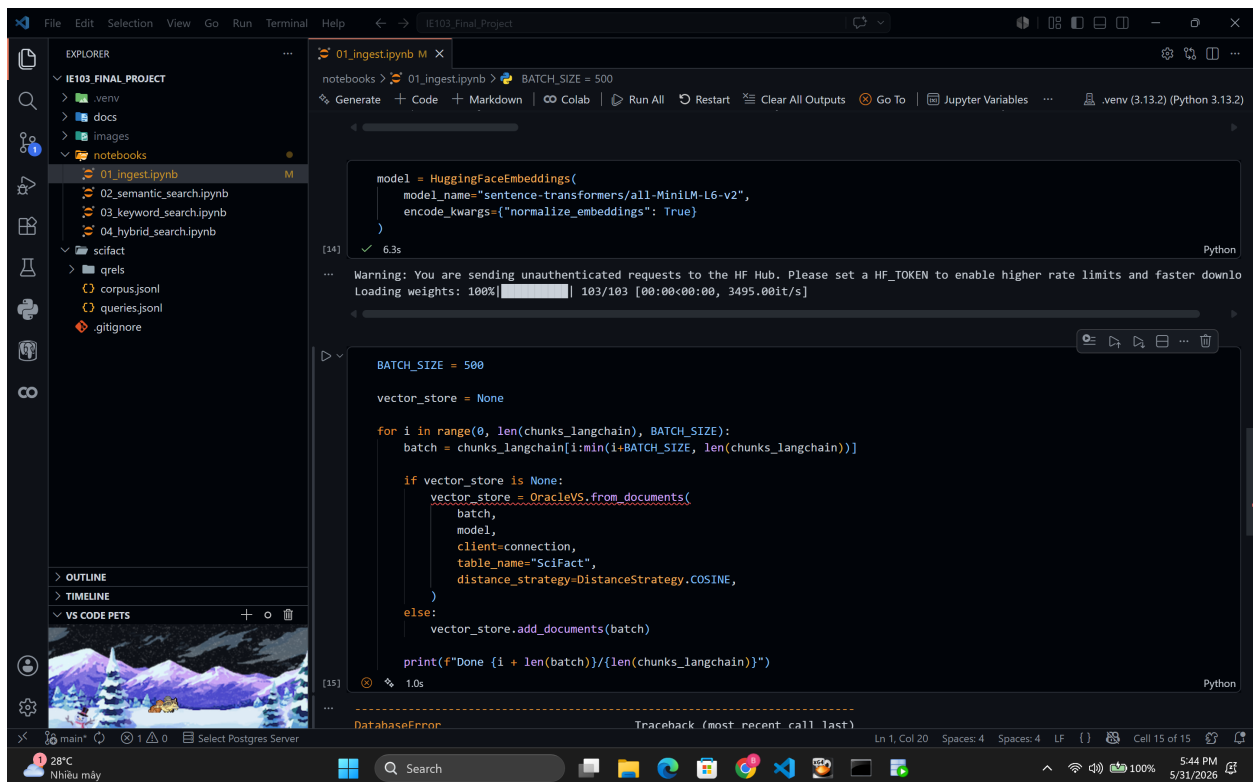


Hình 6.4: Chia nhỏ tài liệu thành các chunks bằng RecursiveCharacterTextSplitter

Sau khi chia nhỏ tài liệu thành các chunk thì các chunk có dạng như sau:

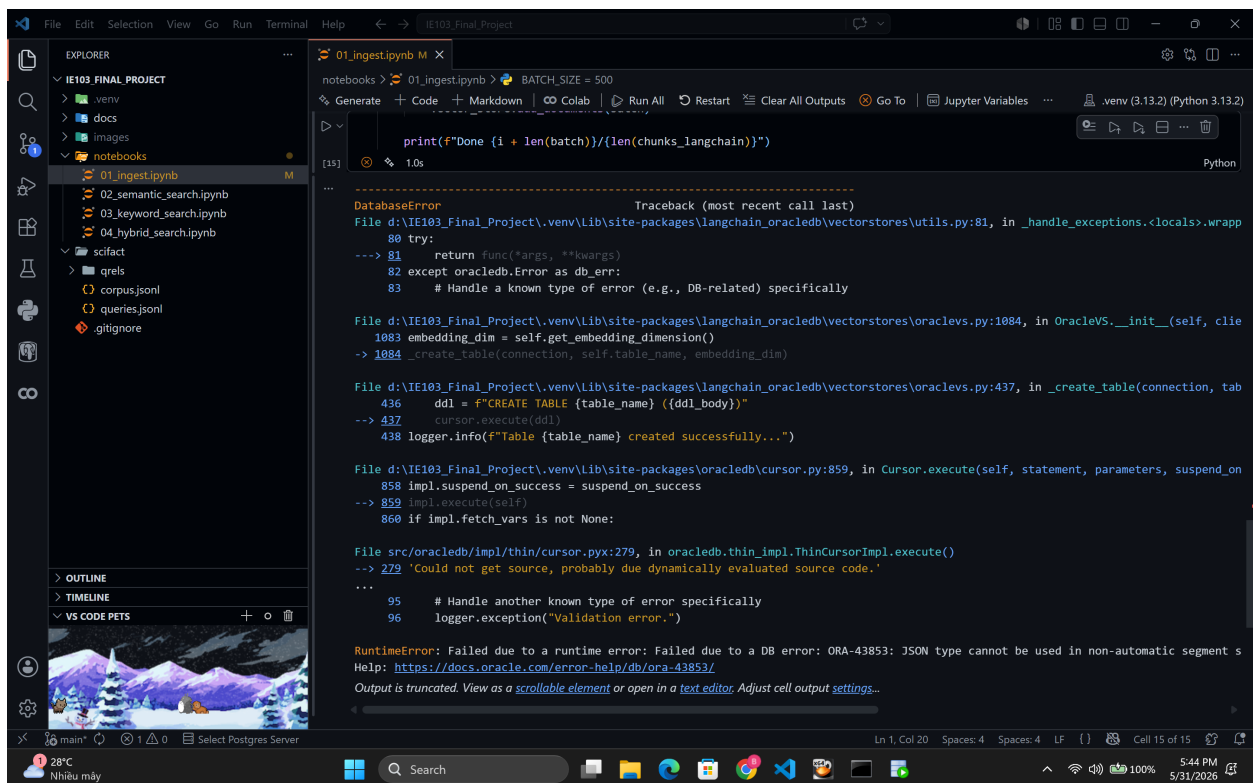
```
1 Document(metadata={'id': '9cd281b5-6b98-4d54-8756-d0582fcc3258', 'title':
    ↳ 'Microstructural development of human newborn cerebral white matter
    ↳ assessed in vivo by diffusion tensor magnetic resonance imaging.',
    ↳ 'corpus_id': '4983'}, page_content='Alterations of the architecture
    ↳ of cerebral white matter in the developing human brain can affect
    ↳ cortical development and result in functional disabilities. A line
    ↳ scan diffusion-weighted magnetic resonance imaging (MRI) sequence
    ↳ with diffusion tensor analysis was applied to measure the apparent
    ↳ diffusion coefficient, to calculate relative anisotropy, and to
    ↳ delineate three-dimensional fiber architecture in cerebral white
    ↳ matter in preterm (n = 17) and full-term infants (n = 7). To assess')
```

Tiếp theo ta sẽ chọn embedding model để embed chunk thành vector vào vector store trong Oracle 26ai Database. Ở đây ta sẽ thực nghiệm với embedding model all-MiniLM-L6-v2, nó ánh xạ các câu và đoạn văn vào không gian vector 384 chiều và có thể được sử dụng cho các tác vụ như phân cụm hoặc tìm kiếm ngữ nghĩa. Sau đó ta sẽ đưa vector đã được embedding từ chunk vào vector store với distance strategy là Cosine Similarity theo từng batch 500 để ổn định và tránh tràn RAM như hình 6.5



Hình 6.5: Chọn embedding model và đưa vào vector store theo từng batch 500 chunk

Tuy nhiên quá trình đưa vào vector store xảy ra lỗi như hình 6.6. Đây là lỗi do thư viện langchain_oracledb trong notebook đang cố gắng khởi tạo Vector Store. Trong quá trình này, nó gọi hàm `_create_table` để tự động tạo một bảng lưu trữ các chunk văn bản, metadata và vector embeddings.

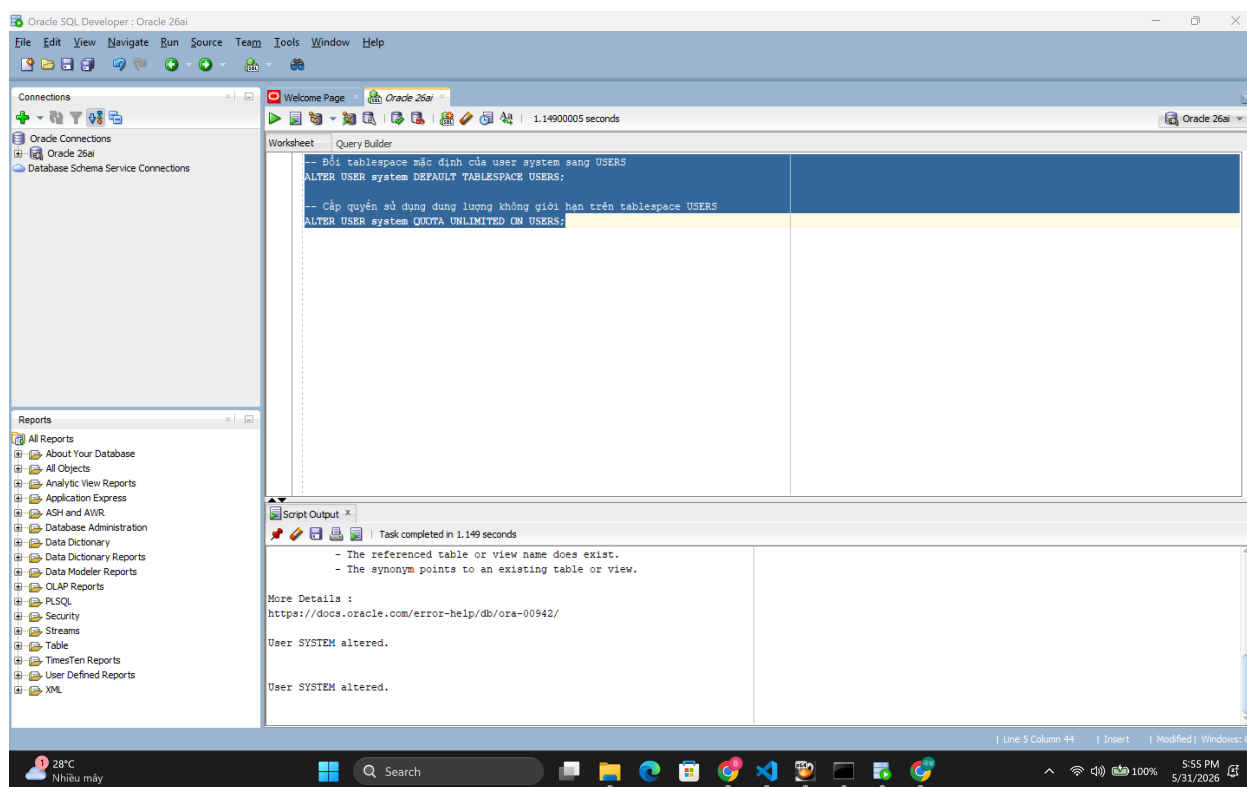


Hình 6.6: Lỗi ASSM khi đưa embedding vector vào vector store

Bảng mà LangChain cố tạo có chứa một cột mang kiểu dữ liệu JSON (thường dùng để lưu metadata). Tuy nhiên, cơ sở dữ liệu Oracle quy định rất chặt chẽ: Bất kỳ bảng nào chứa cột kiểu dữ liệu JSON nguyên bản (native JSON type) đều bắt buộc phải được lưu trữ trong một tablespace có cấu hình Automatic Segment Space Management (ASSM).

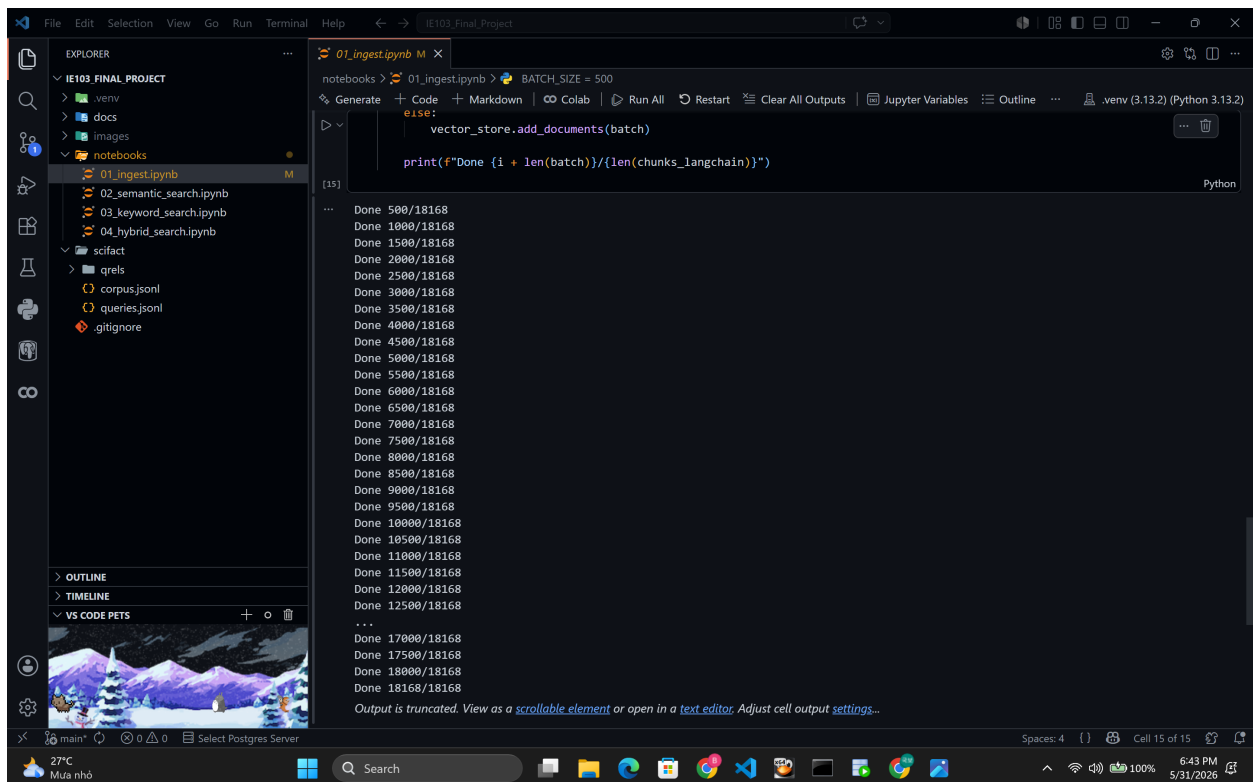
Tablespace mặc định mà user database đang sử dụng hiện tại đang được cấu hình là non-ASSM (Manual).

Để giải quyết vấn đề trên ta chỉ cần chuyển không gian lưu trữ mặc định của tài khoản này sang tablespace **USERS** (tablespace này mặc định đã được cấu hình ASSM sẵn trong Oracle) bằng cách chạy 2 lệnh SQL như hình 6.7:

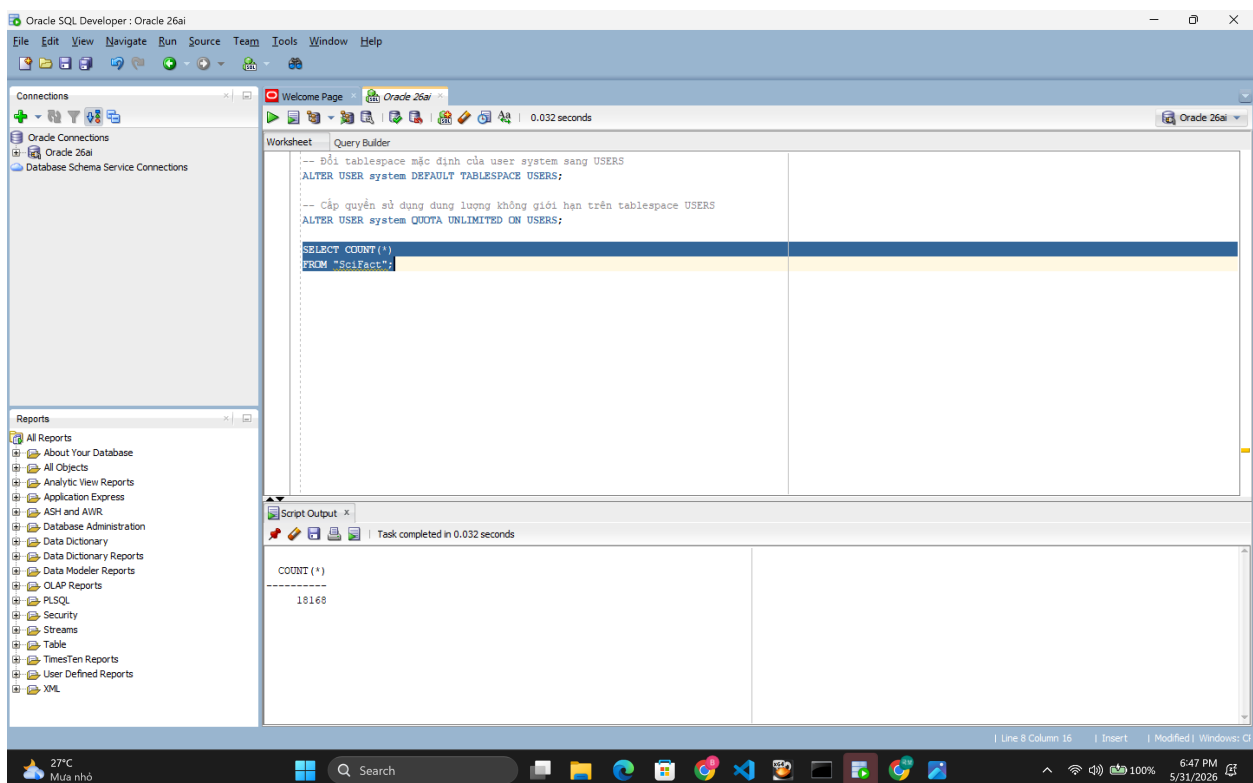


Hình 6.7: Giải quyết lỗi ASSM khi khởi tạo vector store

Sau khi sửa, vector store đã được khởi tạo thành công và đưa các vector embedding vào theo từng batch 500 trong notebook như hình 6.8. Để kiểm tra xem vector store đã thực sự được khởi tạo và đưa vào 18168 vector embedding từ chunk chưa, ta kiểm tra trên SQL Developer với câu lệnh kiểm tra đơn giản `SELECT COUNT(*) FROM "SciFact"`. Hình 6.9 cho thấy sau khi sửa lỗi ASSM, vector store đã được khởi tạo với đủ 18168 vector embedding từ các chunk. Notebook thực nghiệm toàn bộ phần đưa dữ liệu vào vectore store thông qua OracleVS ở [01_ingest.ipynb](#)



Hình 6.8: Đưa các vector embedding vào vector store thành công trên notebook

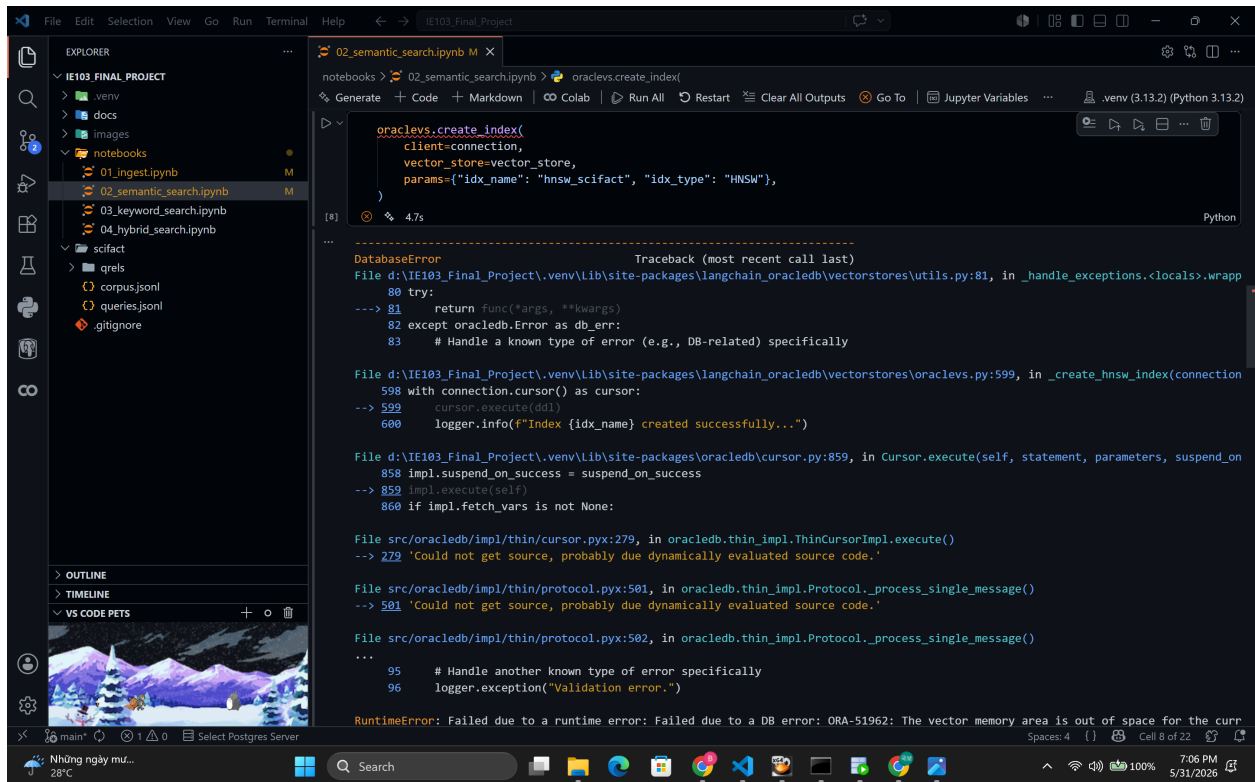


Hình 6.9: Kiểm tra vectore store đã được khởi tạo chưa trên SQL Developer

6.5.2 Tạo HNSW index trên vector store và thực nghiệm Semantic Search

Để tăng tốc độ cho Semantic Search, ta tạo HNSW index trên vector store đã có. Tuy nhiên quá trình tạo có thể gặp lỗi như hình 6.10. Để tạo được HNSW Index (Hierarchical

Navigable Small World - thuật toán tìm kiếm vector rất nhanh nhưng tốn RAM), Oracle cần xây dựng một đồ thị đa tầng trong bộ nhớ (RAM) trước khi lưu nó xuống ổ cứng. Oracle quản lý vùng RAM này bằng một tham số hệ thống tên là `vector_memory_size`. Hiện tại, vùng nhớ này đang bị đầy hoặc chưa được cấp phát đủ dung lượng để chứa đồ thị vector của LangChain đẩy vào.



```
File Edit Selection View Go Run Terminal Help
IE103_Final_Project

EXPLORER
IE103_FINAL_PROJECT
  .env
  docs
  images
  notebooks
    01_ingest.ipynb
    02_semantic_search.ipynb
    03_keyword_search.ipynb
    04_hybrid_search.ipynb
  scifact
    qrels
    corpus.jsonl
    queries.jsonl
    gitignore

OUTLINE
TIMELINE
VS CODE PETS

02_semantic_search.ipynb M
notebooks > 02_semantic_search.ipynb > oraclevs.create_index
Generate + Code + Markdown Colab Run All Restart Clear All Outputs Go To Jupyter Variables .env (3.13.2) (Python 3.13.2)

oraclevs.create_index(
    client=connection,
    vector_store=vector_store,
    params={'idx_name': 'hnsw_scifact', 'idx_type': 'HNSW'},
)

[8] 4.7s Python

DatabaseError Traceback (most recent call last)
File d:\IE103_Final_Project\.env\lib\site-packages\langchain_oracledb\vectors\torstores\utils.py:81, in _handle_exceptions.<locals>.wrap
80 try:
--> 81     return Func(*args, **kwargs)
82 except OracleDBError as db_err:
83     # Handle a known type of error (e.g., DB-related) specifically

File d:\IE103_Final_Project\.env\lib\site-packages\langchain_oracledb\vectors\torstores\oraclevs.py:599, in _create_hnsw_index(connection
598 with connection.cursor() as cursor:
--> 599     cursor.execute(ddl)
600     logger.info(f"Index {idx_name} created successfully...")

File d:\IE103_Final_Project\.env\lib\site-packages\oracledb\cursor.py:859, in Cursor.execute(self, statement, parameters, suspend_on
858 impl.suspend_on_success = suspend_on_success
--> 859 impl.execute(self)
860 if impl.fetch_vars is not None:

File src\oracledb\impl\thin\cursor.py:279, in OracleDBThinImpl.ThinCursorImpl.execute()
--> 279 'Could not get source, probably due dynamically evaluated source code.'

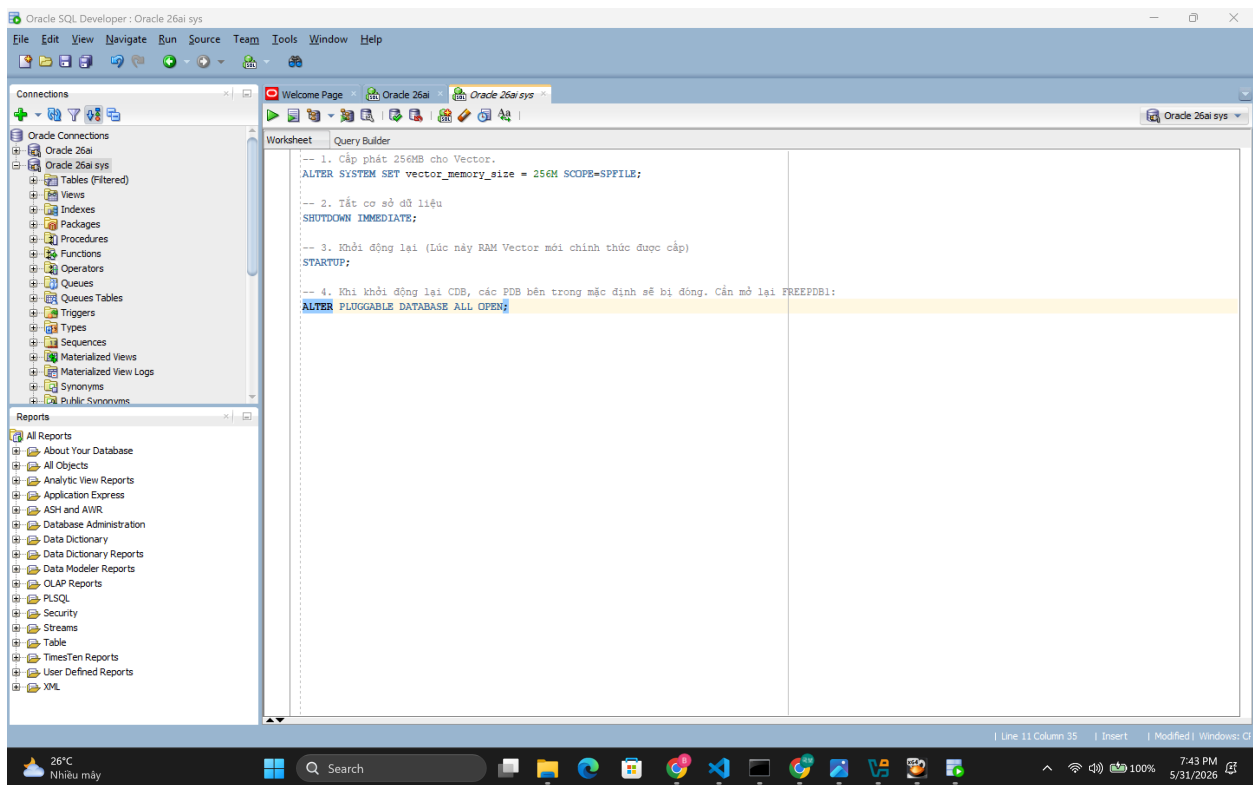
File src\oracledb\impl\thin\protocol.py:501, in OracleDBThinImpl.Protocol._process_single_message()
--> 501 'Could not get source, probably due dynamically evaluated source code.'

File src\oracledb\impl\thin\protocol.py:502, in OracleDBThinImpl.Protocol._process_single_message()
...
95 # Handle another known type of error specifically
96 logger.exception("Validation error.")

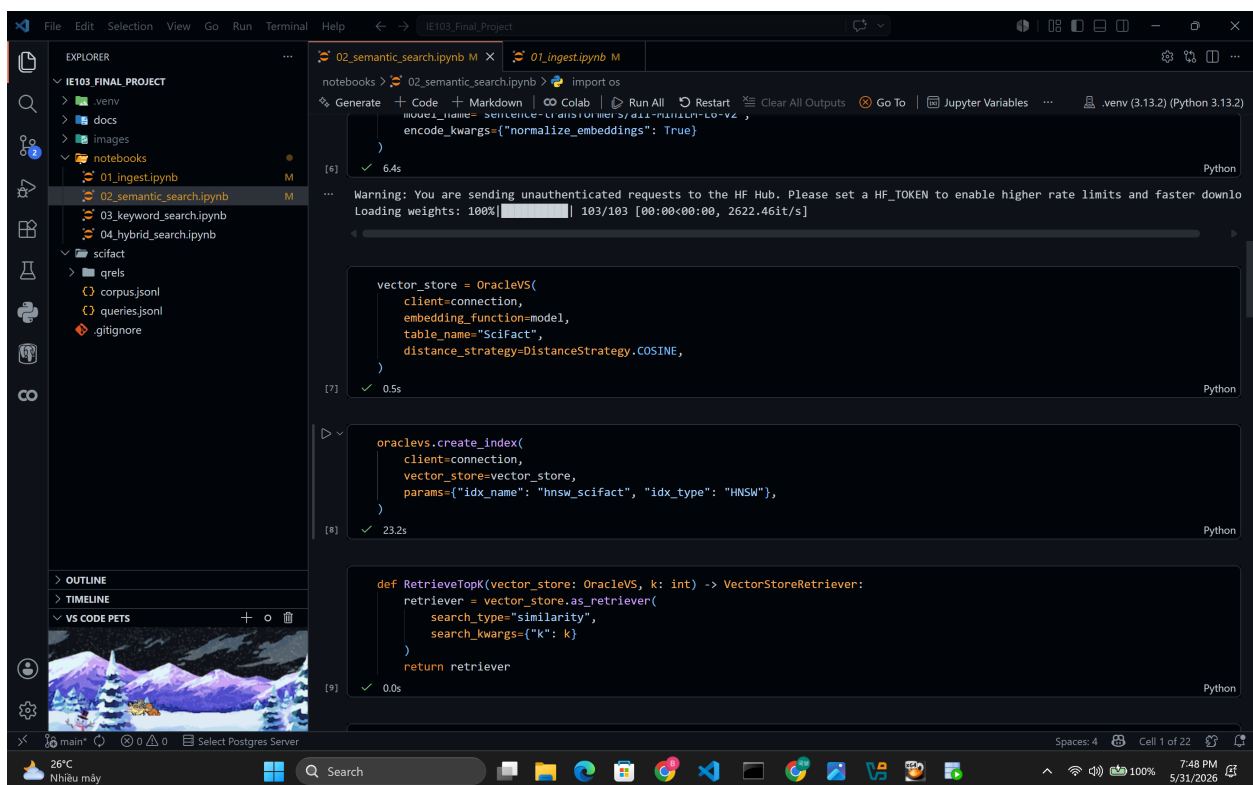
RuntimeError: Failed due to a runtime error: Failed due to a DB error: ORA-51962: The vector memory area is out of space for the current operation.
```

Hình 6.10: Lỗi khi tạo HNSW index

Để xử lý vấn đề trên, đầu tiên ta cần kết nối mới tới Oracle 26ai Database bằng SID `free` thay vì Service name `FREEPDB1`. Sau khi đã kết nối tới SID `free` trên SQL Developer, chạy 4 câu lệnh SQL như hình 6.11 để cấp cho `vector memory area` 256MB dung lượng lưu trữ. Sau khi chạy thành công, ta khởi động lại Oracle 26ai Database trên VirtualBox cũng như kết nối tới Oracle 26ai Database thông qua Service name `FREEPDB1` trên SQL Developer. Sau khi xong, cell tạo HNSW đã chạy bình thường, bước tiếp theo chính là viết retriever để trích xuất các tài liệu có liên quan khi để phục vụ bài toán xác minh luận điểm khoa học. Ở đây ta dùng `search_type` là `similarity` vì `distance_strategy` là `Cosine similarity` và lấy ra top k chunk có liên quan tới luận điểm khoa học, sau đó lấy ra tài liệu gốc từ chunk. Hình 6.12 cho thấy HNSW index đã được tạo thành công và retriever để lấy ra top k chunk có liên quan tới luận điểm khoa học dựa trên ngữ nghĩa



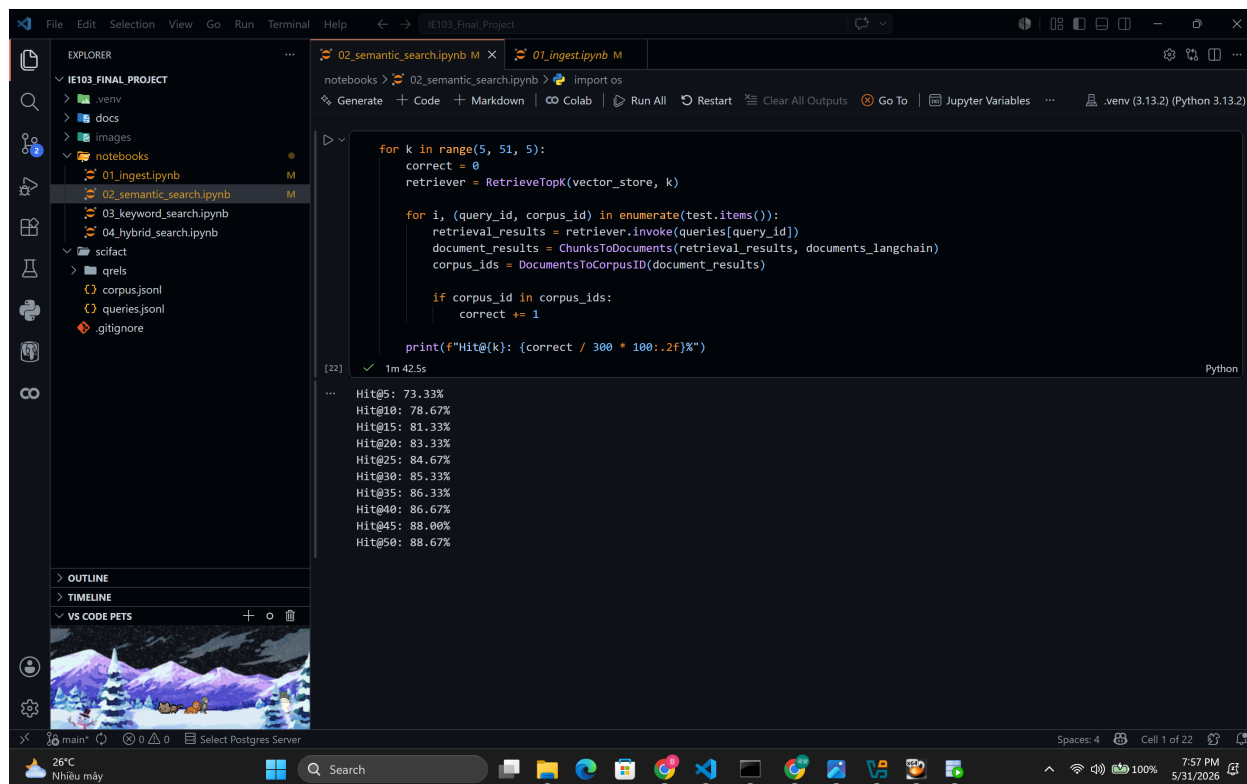
Hình 6.11: Sửa lỗi vector memory area



Hình 6.12: Tạo HNSW index thành công và viết retriever top k chunk dựa trên ngữ nghĩa

Sau khi đã có retriever, ta viết thêm các hàm để lấy tài liệu gốc từ chunk và các hàm tính Hit@k từ tập test của bộ dữ liệu. Hit@k ở đây là một metric đánh giá hệ thống tìm kiếm/retrieval dựa trên ý tưởng rất đơn giản: nếu đáp án đúng xuất hiện trong top k kết quả

trả về thì tính là hit = 1, ngược lại hit = 0. Sau đó lấy trung bình trên tất cả các query. Kết quả Semantic Search với Cosine similarity trên vector store có HNSW index với trong hình 6.13. Ta chọn k trong đoạn [5, 50] với bước nhảy 5 để xem được sự thay đổi của Hit@k khi k tăng lên. Thời gian retrieve 300 query trên tập test của Semantic Search khá nhanh và cho kết quả khá cao khi Hit@5 đã đạt 73.33% và Hit@50 đạt đến 88.67%. Notebook thực nghiệm toàn bộ phần Semantic Search ở [02_semantic_search.ipynb](#)



```
for k in range(5, 51, 5):
    correct = 0
    retriever = RetrieveTopK(vector_store, k)

    for i, (query_id, corpus_id) in enumerate(test.items()):
        retrieval_results = retriever.invoke(queries[query_id])
        document_results = ChunksToDocuments(retrieval_results, documents_langchain)
        corpus_ids = DocumentsToCorpusID(document_results)

        if corpus_id in corpus_ids:
            correct += 1

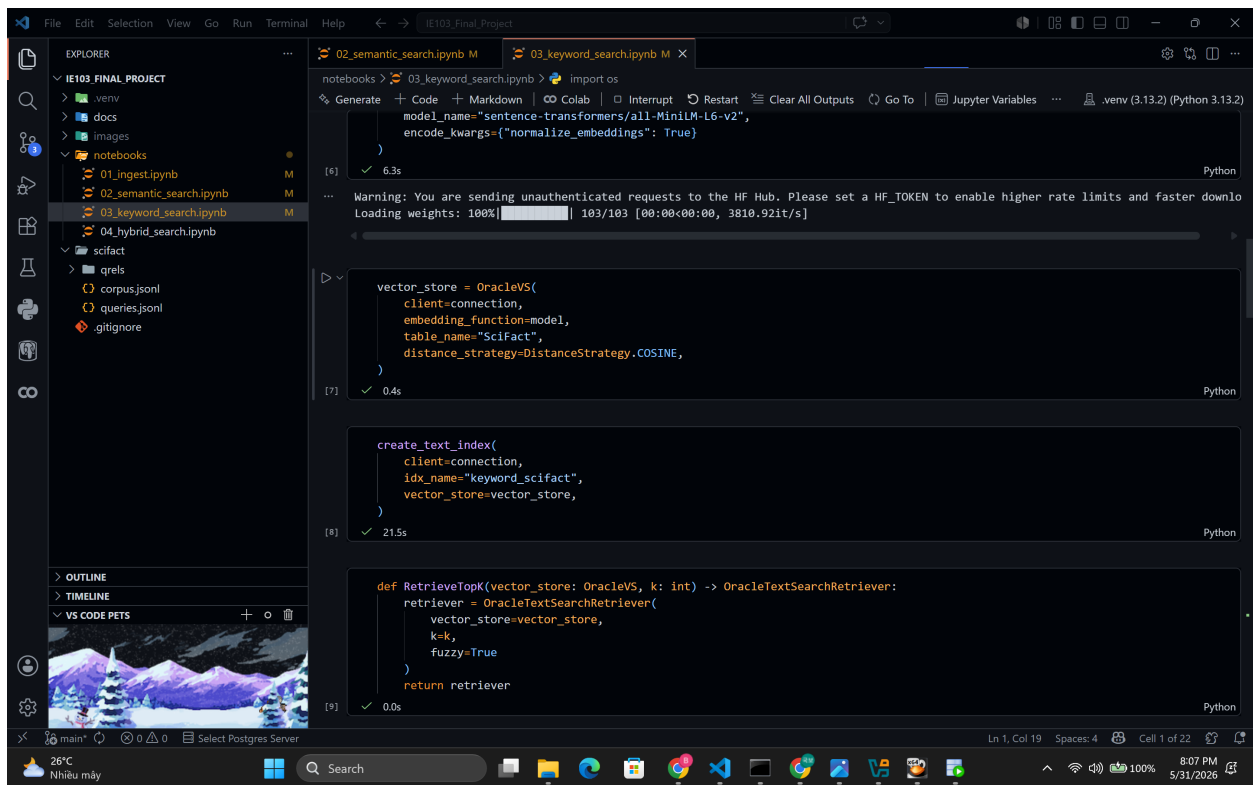
    print(f"Hit@{k}: {correct / 300 * 100:.2f}%")
```

Hit@5: 73.33%
Hit@10: 78.67%
Hit@15: 81.33%
Hit@20: 83.33%
Hit@25: 84.67%
Hit@30: 85.33%
Hit@35: 86.33%
Hit@40: 86.67%
Hit@45: 88.00%
Hit@50: 88.67%

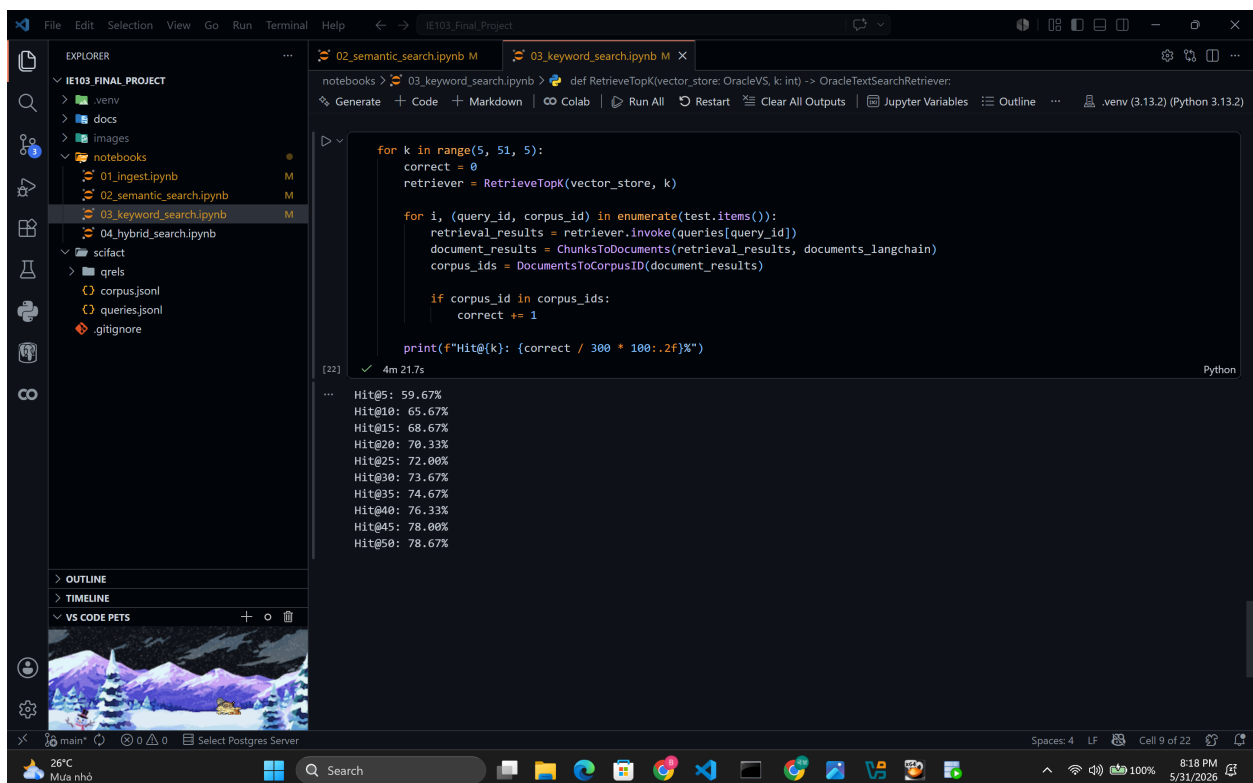
Hình 6.13: Kết quả thực nghiệm Semantic Search trên tập test

6.5.3 Tạo text index trên vector store và thực nghiệm Keyword Search

Ta sẽ tạo text index phục vụ cho Keyword Search/Oracle full text search và viết retriever lấy ra top k chunk tương tự phần Semantic Search. Ở đây `OracleTextSearchRetriever` có thêm tham số `fuzzy=True` nhằm xử lý các trường hợp sai lệch chính tả. Cell tạo index và retriever trong hình 6.14 và kết quả thực nghiệm trên tập test trong hình 6.15. Có thể thấy thời gian retrieve 300 query trên tập test của Keyword Search chậm hơn khoảng 2.5 lần so với Semantic Search nhưng kết quả lại khá tệ. Cụ thể, Hit@5 của Keyword Search chỉ đạt 59.67% và Hit@50 của Keyword Search chỉ bằng Hit@10 của Semantic Search. Có thể thấy Semantic Search tốt hơn nhiều so với Keyword Search cả về thời gian truy xuất lẫn độ chính xác. Notebook toàn bộ thực nghiệm phần Keyword Search ở [03_keyword_search.ipynb](#)



Hình 6.14: Tạo text index và retriever Keyword Search



Hình 6.15: Kết quả thực nghiệm Keyword Search trên tập test

6.5.4 Thực nghiệm Hybrid Search và nhận xét

Tuy Keyword Search sẽ cho độ chính xác cao với từ khóa cụ thể, nhưng lại không thể hiểu ngữ nghĩa và có thể bỏ sót tài liệu nếu dùng từ đồng nghĩa. Semantic Search lại có thể hiểu

ngữ nghĩa câu truy vấn, tìm được tài liệu liên quan dù có thể khác từ ngữ, tuy vậy vẫn có thể trả về kết quả không chứa từ khóa quan trọng cần truy vấn. Vì vậy, việc kết hợp hai cách search thành Hybrid Search có khả năng giải quyết được các vấn đề trên, vì Keyword Search đảm bảo các từ khóa quan trọng được ưu tiên và Semantic Search giúp tìm thêm các tài liệu liên quan về mặt ngữ nghĩa. Do đó ta tiếp tục thực nghiệm nghiêm ngặt trên toàn bộ 300 câu truy vấn thuộc tập kiểm thử (test set) của SciFact.

```

for k in range(5, 51, 5):
    correct = 0
    semantic_retriever = SemanticRetrieveTopK(vector_store, k)
    keyword_retriever = KeywordRetrieveTopK(vector_store, k)

    for i, (query_id, corpus_id) in enumerate(test.items()):
        semantic_retrieval_results = semantic_retriever.invoke(queries[query_id])
        semantic_document_results = ChunksToDocuments(semantic_retrieval_results, documents_langchain)
        semantic_corpus_ids = DocumentsToCorpusID(semantic_document_results)

        keyword_retrieval_results = keyword_retriever.invoke(queries[query_id])
        keyword_document_results = ChunksToDocuments(keyword_retrieval_results, documents_langchain)
        keyword_corpus_ids = DocumentsToCorpusID(keyword_document_results)

        corpus_ids = set(semantic_corpus_ids + keyword_corpus_ids)

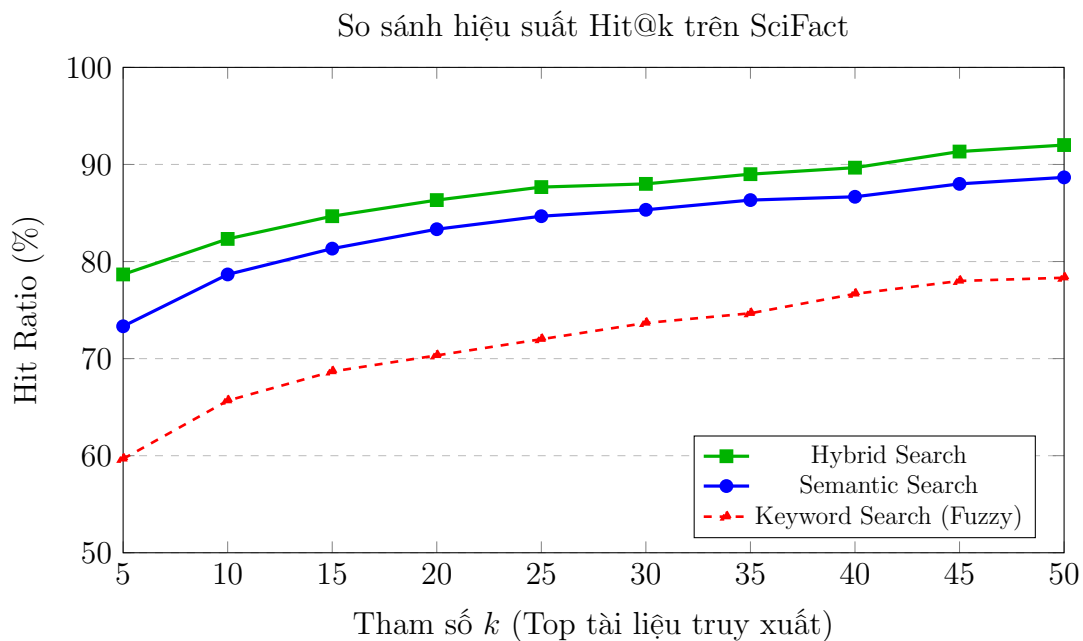
        if corpus_id in corpus_ids:
            correct += 1

    print(f"Hit@{k}: {correct / 300 * 100:.2f}%")

```

Hit@5: 78.67%
 Hit@10: 82.33%
 Hit@15: 84.67%
 Hit@20: 86.33%
 Hit@25: 87.67%
 Hit@30: 88.00%
 Hit@35: 89.00%
 Hit@40: 89.67%
 Hit@45: 91.33%
 Hit@50: 92.00%

Hình 6.16: Kết quả thực nghiệm Hybrid Search trên tập test



Hình 6.17: Xu hướng thay đổi hiệu suất truy xuất khi tăng tham số k.

Dựa vào kết quả thu được tại hình 6.16-6.17, Semantic Search thể hiện sự vượt trội rõ rệt so với Keyword Search truyền thống, với mức chênh lệch hiệu suất luôn duy trì từ 10% đến hơn 13% ở tất cả các mốc k . Điều này chứng minh rằng trong miền tri thức khoa học, việc nắm bắt ngữ cảnh và ý niệm chuyên sâu đóng vai trò rất quan trọng.

Đáng chú ý, phương pháp Hybrid Search đạt được kết quả tối ưu nhất ở mọi cấu hình thực nghiệm dù phải đánh đổi một chút thời gian chạy. Tại mốc **Hit@10**, Hybrid Search giúp nâng hiệu suất lên **82.33%**. Khi mở rộng không gian tìm kiếm lên **Hit@50**, mô hình lại đạt tỷ lệ chính xác ấn tượng lên đến **92.00%**. Kết quả này chứng minh tính bổ trợ mạnh mẽ giữa cơ chế so khớp từ vựng chính xác của Oracle Text và cơ chế hiểu ngữ nghĩa của AI Vector Search.

Thực nghiệm đã chứng minh tính hiệu quả vượt trội khi ứng dụng hệ quản trị cơ sở dữ liệu thế hệ mới Oracle 26ai kết hợp thư viện LangChain để giải quyết bài toán truy xuất thông tin trên tập dữ liệu khoa học phức tạp SciFact. Giải pháp Hybrid Search tích hợp trực tiếp trong một hệ thống lưu trữ duy nhất không chỉ đơn giản hóa kiến trúc hạ tầng dữ liệu RAG mà còn đem lại độ chính xác cực kỳ cao.